

COL380 Assignment 1

Design Decisions Report

Popat Nihal Alkesh
2023CS10058

1 updateDisplay

Design: Parallelization occurs in three stages. (1) Snapshot construction: each thread independently builds local `unordered_map` structures using `#pragma omp parallel for`. (2) Sequential propagation: buy/sell prices carry forward across snapshots, creating a temporal dependency handled in a sequential loop. (3) Sorting and output: parallel sorting (by spread, then stock ID) and file writing. Dynamic scheduling proved more effective through experimentation, likely due to load imbalance across snapshots.

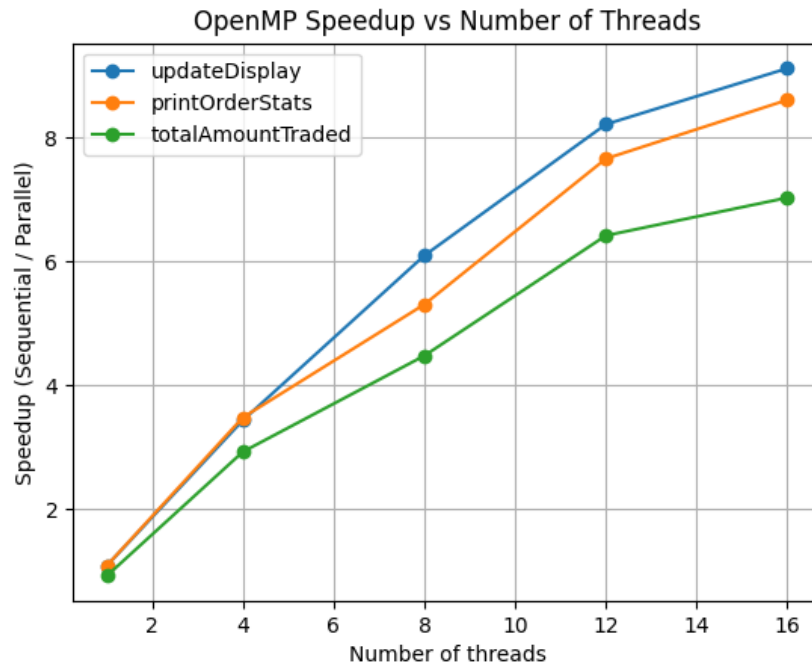
2 totalAmountTraded

Design: This embarrassingly parallel problem sums `orderQty × orderValue` across all packets. Implementation uses `#pragma omp parallel for reduction(+:total)`, where the reduction clause automatically handles synchronization. Bit unstuffing overhead is minimal and doesn't prevent near-linear speedup up to the physical core count.

3 printOrderStats

Design: Each thread maintains a private `unordered_map` for thread-local statistics accumulation, eliminating contention during packet processing. A critical section merges thread-local maps into a global array (indexed by stock ID, bounded at 10^6). Output is generated in increasing stock ID order, skipping inactive stocks. Most work happens in parallel; only the merge step requires synchronization.

4 Performance Analysis



As we can see, the graphs are almost linear, and slowly getting horizontal. This makes sense since, as number of threads increase, each thread gets less resource, since we have limited (12 in this case) cores.